



南 開 大 學
Nankai University

南 开 大 学

计 算 机 学 院

计算机系统前沿实验报告

存储设备性能测试和分析

王 烨

年级：2024 级

专业：计算机科学与技术

指导教师：崔立骁

2026 年 4 月 4 日

目录

一、 测试环境说明	1
二、 测试方法	1
三、 测试流程	1
(一) 基准测试	1
(二) 多维度参数测试	2
1. 块大小 (bs) 的影响	2
2. 并发与队列深度 (numjobs and iodepth) 的影响	7
3. I/O 引擎 (ioengine) 的对比	8
四、 测试分析及结论	10
(一) 基准测试下读写方式性能差异/相近的原因	10
(二) 块大小对吞吐量和 IOPS 的影响	10
1. 顺序读 (read) 下块大小 (bs) 从 4k 升高导致的带宽 (BW) 断崖下降	10
2. 顺序读 (read) 下块大小 (bs) 在 8k+ 时带宽 (BW) 的异常稳定	10
(三) 并发 (numjobs) 与队列深度 (iodepth) 对性能的影响	11
1. 对顺序读和随机写的影响	11
2. 对顺序写的影响	11
3. 对随机读的影响	11
(四) I/O 引擎对性能的影响	11
五、 总结	12

一、 测试环境说明

- CPU: 13th Gen Intel(R) Core(TM) i7-13650HX
- 内存: 11 GB (基于 WSL2)
- 操作系统: Ubuntu 22.04 LTS
- 存储设备: NVMe SSD

二、 测试方法

本实验采用控制变量法, 首先设定基准测试, 通过 fio 编写测试脚本, 通过 WSL2 配置的 Ubuntu 22.04 LTS 开展对原机 NVMe SSD 存储性能的测试, 之后控制单个参数变量, 逐项开展对比测试, 记录 IOPS、带宽 (BW) 和延迟 (latency)。

由于本实验侧重于参数变化下的性能对比, 以及由于实际测试环境的限制因素, WSL2 本身的轻量虚拟化导致与原生 Linux 的性能差距不再加以讨论。

三、 测试流程

(一) 基准测试

首先编写用于基准测试的 fio 脚本, 随后开展实验:

storage-test

```
1 [global]
2 filename=testfile
3 size=5G
4 runtime=30
5 direct=1
6 rw=read
7
```

```
8 ===== 任务1: 基础性能测试 =====
```

```
9
10 [1-seqread]
11 rw=read
12 bs=4k
13 ioengine=pvsync
14 numjobs=1
15 iodepth=1
16
17 [1-seqwrite]
18 rw=write
19 bs=4k
20 ioengine=pvsync
21 numjobs=1
22 iodepth=1
23
```

```

24 [1-randread]
25 rw=randread
26 bs=4k
27 ioengine=pvsync
28 numjobs=1
29 iodepth=1
30
31 [1-randwrite]
32 rw=randwrite
33 bs=4k
34 ioengine=pvsync
35 numjobs=1
36 iodepth=1

```

在本次实验中，我们设定基准测试参数为：

块大小 bs=4k, I/O 引擎为 pvsync, 线程数 numjobs=1, I/O 深度 iodepth=1

测试结果如表所示：

表 1: 基准性能测试

读写方式 参数	read	write	randread	randwrite
IOPS	536K	543K	7919	369K
BW	2095MiB/s	2122MiB/s	30.9MiB/s	1411MiB/s
latency	1766.87ns	1554.24ns	124.40us	2475.79ns

(二) 多维度参数测试

1. 块大小 (bs) 的影响

控制其他参数不变，分别设定块大小 bs 为 8k, 16k, 64k, 对四种读写方式分组开展实验。

首先开展顺序读 (read) 实验测试：

表 2: 顺序读时不同块大小下的性能对比

块大小 参数	4k	8k	16k	64k
IOPS	536K	99.1K	50.2K	12.8K
BW	2095MiB/s	774MiB/s	784MiB/s	799MiB/s
latency	1766.87ns	9640.86ns	19148.76ns	75.41us

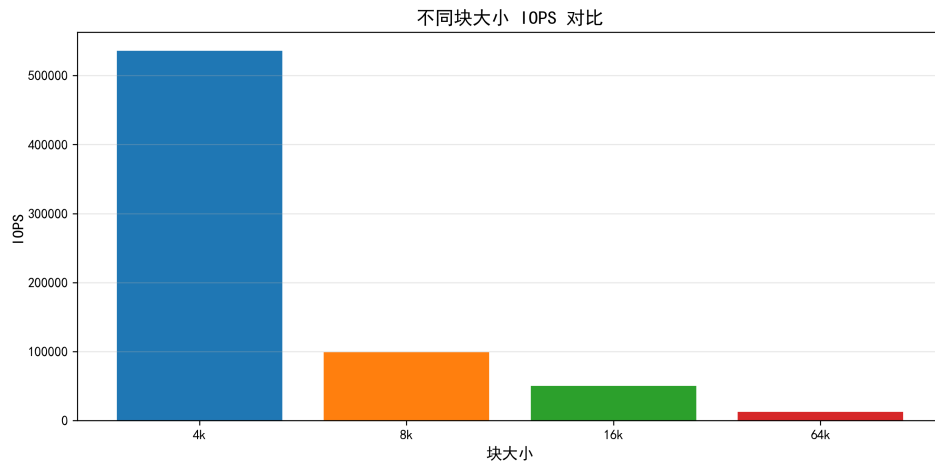


图 1: 不同块大小 IOPS 对比

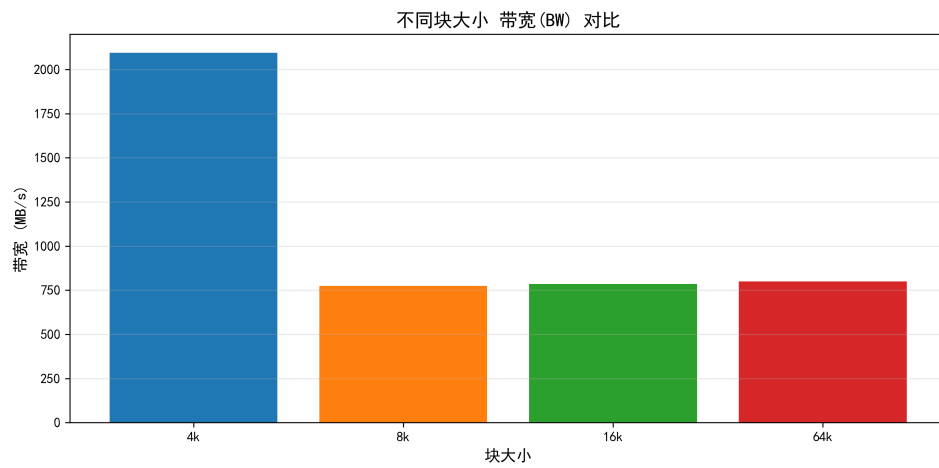


图 2: 不同块大小 BW 对比

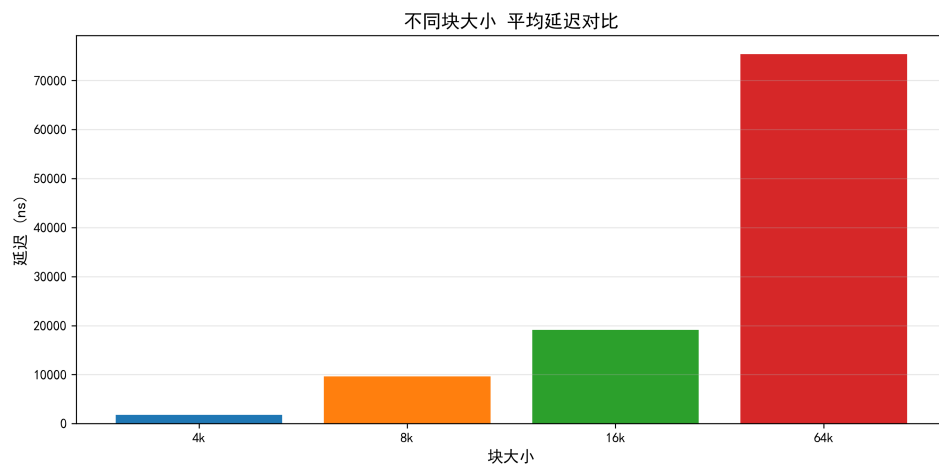


图 3: 不同块大小延迟对比

然后开展顺序写 (write) 实验测试:

表 3: 顺序写时不同块大小下的性能对比

参数 \ 块大小	4k	8k	16k	64k
IOPS	543K	363K	176K	46.7K
BW	2122MiB/s	2838MiB/s	2750MiB/s	2921MiB/s
latency	1554.24ns	2339.91ns	4.60us	17.52us

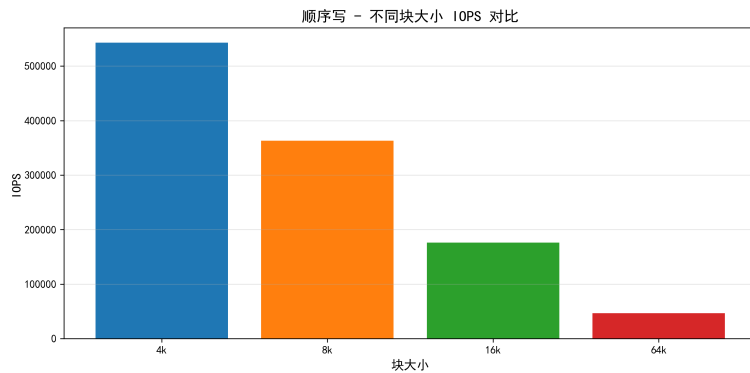


图 4: 不同块大小 IOPS 对比

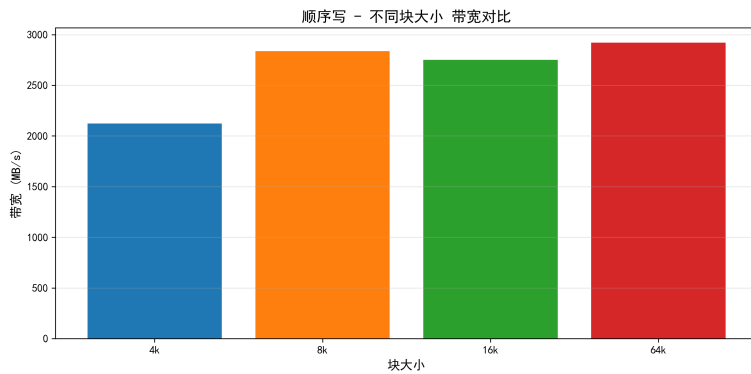


图 5: 不同块大小 BW 对比

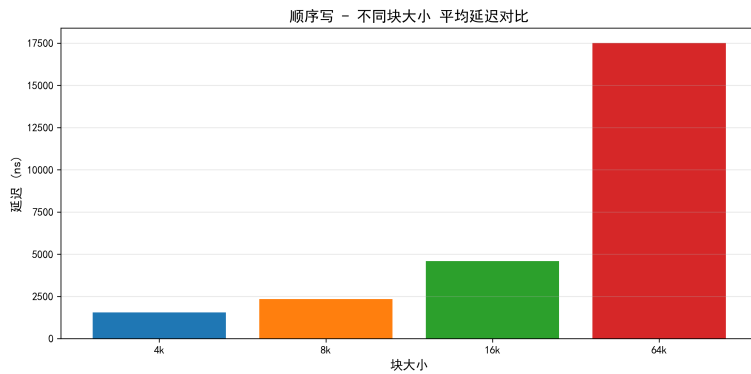


图 6: 不同块大小延迟对比

接着开展随机读（randread）实验测试：

表 4: 随机读时不同块大小下的性能对比

参数 \ 块大小	4k	8k	16k	64k
IOPS	7919	7682	7464	5384
BW	30.9MiB/s	60.0MiB/s	117MiB/s	337MiB/s
latency	124.40us	128.34us	132.99us	182.57us

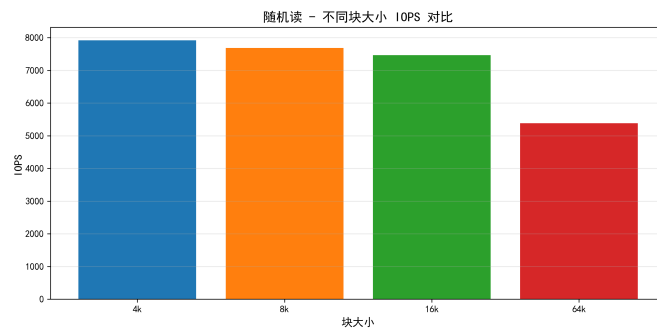


图 7: 不同块大小 IOPS 对比

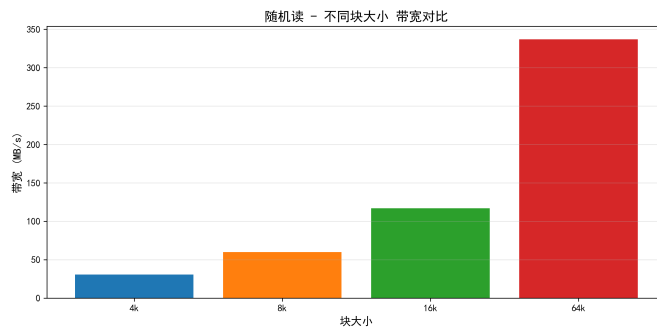


图 8: 不同块大小 BW 对比

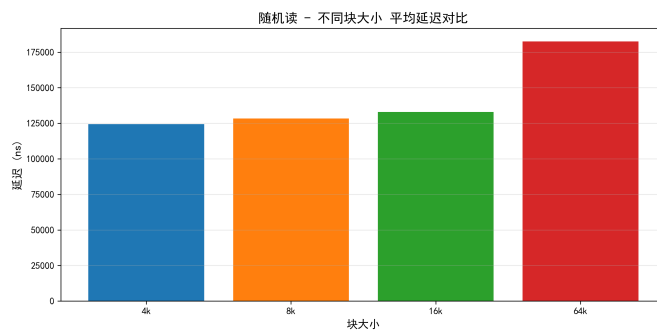


图 9: 不同块大小延迟对比

最后开展随机写（randwrite）实验测试：

表 5: 随机写时不同块大小下的性能对比

参数 \ 块大小	4k	8k	16k	64k
IOPS	369K	238K	137K	40.1K
BW	1411MiB/s	1856MiB/s	2137MiB/s	2505MiB/s
latency	2475.79ns	3639.33ns	5.31us	18.09us

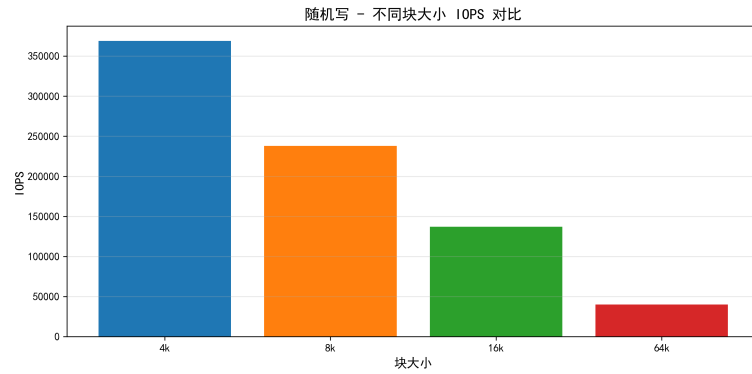


图 10: 不同块大小 IOPS 对比

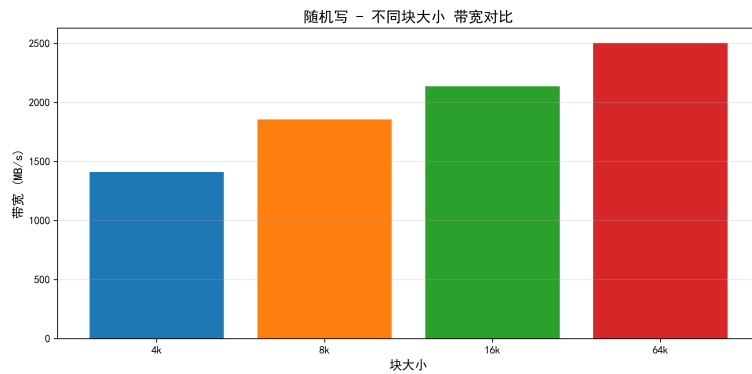


图 11: 不同块大小 BW 对比

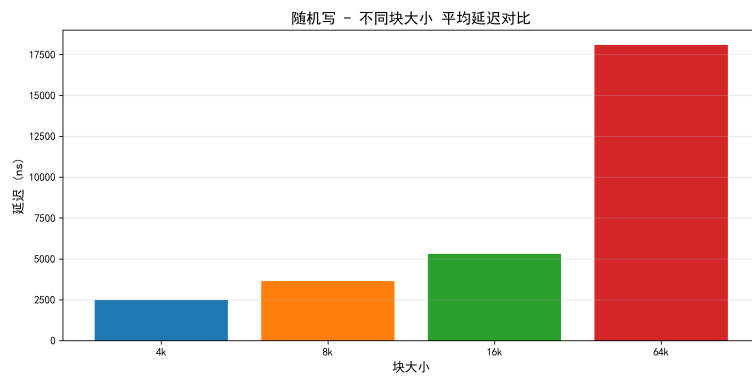


图 12: 不同块大小延迟对比

2. 并发与队列深度 (numjobs and iodepth) 的影响

控制其他参数不变, 设定块大小 bs=4k, 控制 numjobs 和 iodepth 的参数变化, 对四种读写方式分组开展实验。以下是四种读写方式的实验测试:

表 6: 顺序读 (read) 时不同并发数与队列深度下的性能对比

	numjobs=4		numjobs=8	
	iodepth=16	iodepth=64	iodepth=16	iodepth=64
IOPS	384.8K	424K	297.2K	314.4K
BW	1502MiB/s	1653MiB/s	1198MiB/s	1229MiB/s
Latency	9919.39ns	9053.28ns	25488.16ns	24881.88ns

表 7: 顺序写 (write) 时不同并发数与队列深度下的性能对比

	numjobs=4		numjobs=8	
	iodepth=16	iodepth=64	iodepth=16	iodepth=64
IOPS	1092K	1112K	1160K	1224K
BW	4264MiB/s	4340MiB/s	4531MiB/s	4781MiB/s
Latency	3545.31ns	3487.53ns	6377.01ns	6045.22ns

表 8: 随机读 (randread) 时不同并发数与队列深度下的性能对比

	numjobs=4		numjobs=8	
	iodepth=16	iodepth=64	iodepth=16	iodepth=64
IOPS	42.8K	44.4K	397.6K	416K
BW	167MiB/s	173MiB/s	1550MiB/s	1622MiB/s
Latency	92016.07ns	89665.35ns	19580.09ns	18722.01ns

表 9: 随机写 (randwrite) 时不同并发数与队列深度下的性能对比

	numjobs=4		numjobs=8	
	iodepth=16	iodepth=64	iodepth=16	iodepth=64
IOPS	330.8K	340.8K	320K	260K
BW	1292MiB/s	1315MiB/s	1248MiB/s	1017MiB/s
Latency	11770.05ns	11419.78ns	24192.83ns	30271.23ns

3. I/O 引擎 (ioengine) 的对比

保持其他参数不变，通过改变 I/O 引擎对比测试存储设备的性能。基于 Linux 系统，我们可以使用的 I/O 引擎有 pvsync, libaio, io-uring, mmap。

表 10: 顺序读 (read) 时不同 I/O 引擎下的性能对比

参数 \ I/O 引擎	pvsync	libaio	io-uring	mmap
IOPS	139K	141K	144K	138K
BW	544MiB/s	550MiB/s	562MiB/s	538MiB/s
latency	7059.81ns	6813.25ns	6645.51ns	6861.00ns

表 11: 顺序写 (write) 时不同 I/O 引擎下的性能对比

参数 \ I/O 引擎	pvsync	libaio	io-uring	mmap
IOPS	560K	493K	27.3K	203K
BW	2186MiB/s	1927MiB/s	107MiB/s	792MiB/s
latency	1489.37ns	1658.35ns	35.91us	4396.93ns

表 12: 随机读 (randread) 时不同 I/O 引擎下的性能对比

参数 \ I/O 引擎	pvsync	libaio	io-uring	mmap
IOPS	7922	7277	7145	6919
BW	30.9MiB/s	28.4MiB/s	27.9MiB/s	27.0MiB/s
latency	124.27us	136.65us	139.09us	143.69us

表 13: 随机写 (randwrite) 时不同 I/O 引擎下的性能对比

参数 \ I/O 引擎	pvsync	libaio	io-uring	mmap
IOPS	337K	267K	25.8K	6315
BW	1318MiB/s	1042MiB/s	101MiB/s	24.7MiB/s
latency	2742.10ns	2920.15ns	37.38us	155.36us

通过折线图，我们可以更直观地看出各 I/O 引擎对四种读写方式的性能差异：

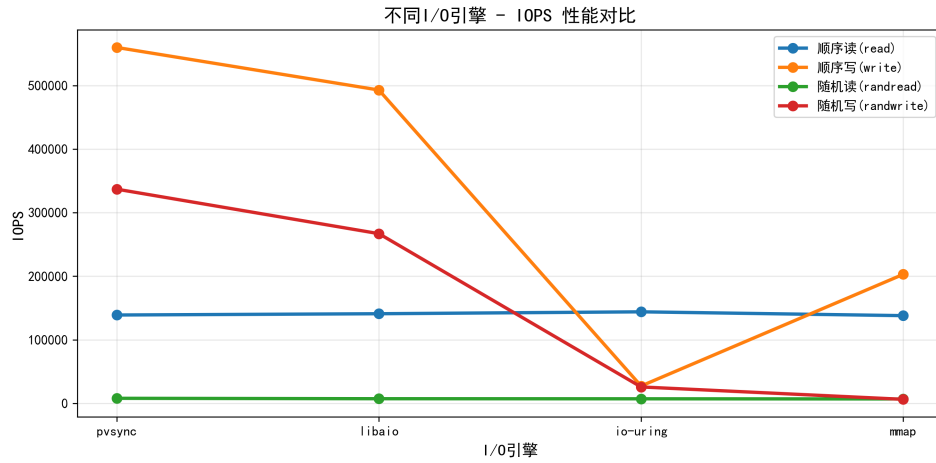


图 13: 不同 I/O 引擎的 IOPS 对比

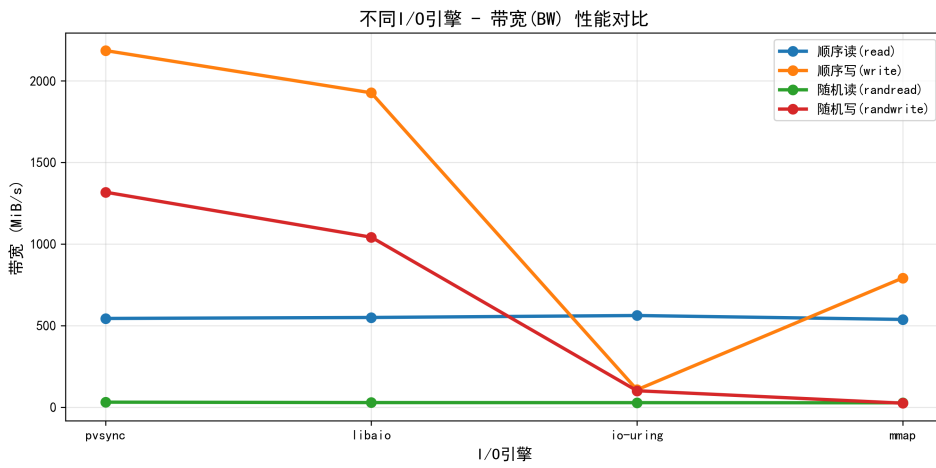


图 14: 不同 I/O 引擎的 BW 对比

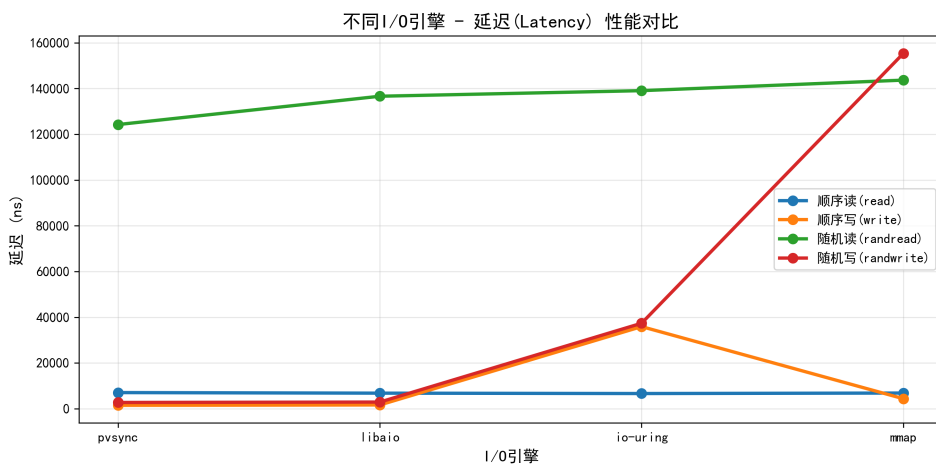


图 15: 不同 I/O 引擎的延迟对比

四、 测试分析及结论

(一) 基准测试下读写方式性能差异/相近的原因

在基准测试中，我们不难注意到两个关键差异：顺序读写的性能远高于随机读写；随机写的性能远高于随机读。在 $\text{numjobs}=1$ 和 $\text{iodepth}=1$ 的前提下，NVMe SSD 无法利用并行优势，其性能只依赖于寻址开销和单次 I/O 延迟。

对于第一个问题，其核心在于 NVMe SSD 的硬件构成和 FTL 映射机制。NVMe SSD 由 NAND Flash 颗粒组成，其以页（page）为单位读写，以块（block）为单位擦除。顺序读写时由于地址连续，控制器可以整页和整 Die 连续操作，而随机读写时存在地址跳变，每次需要重新寻址，带来了巨大的寻址开销，严重限制了存储性能。此外，顺序读写时 FTL 可以连续映射，随机读写时 FTL 频繁查表，随机映射，在单线程条件下也带来了巨大的延迟。

对于第二个问题，主要原因在于基准测试的设定和 NVMe SSD 的随机读写机制。基准测试设定了 $\text{iodepth}=1$ ，这锁死了程序只能通过串行开展指令。此时 NVMe SSD 的写路径为异步写入缓冲，主机发送随机写指令，SSD 控制器接受数据后立刻写入 DRAM 或 SLC cache，并同时向主机反馈完成。此时用户端所见的延迟实际为缓存的写入延迟，而数据从缓存搬出的后续操作延迟被隐藏。而读路径为强同步操作，主机发送随机读指令，SSD 控制器查阅 FTL 映射表，寻址后选取通道，一步步到物理页后启动 NAND 开展读取，最后将数据传回主机，用户端所见的延迟即为真实的闪存物理延迟。当 iodepth 拉高后，并行化开始发挥作用，随机读的性能会逐渐反超随机写。

(二) 块大小对吞吐量和 IOPS 的影响

注意到当块大小（bs）从 4k 逐渐升高到 64k 时，对于四种读写方式，从普遍规律下，IOPS 逐渐降低，吞吐量（BW）逐渐升高至趋于饱和，这是因为 $\text{IOPS} = \text{每秒完成的 I/O 次数}$ ， $\text{BW} = \text{IOPS} \times \text{bs}$ 。

当 bs 升高时，单次传输的数据变多，单次 I/O 的处理时间变长，IOPS 随之降低；同时，bs 的升高和 IOPS 的降低影响相互平衡，使 BW 逐渐逼近硬件限制的极限带宽，对于顺序写，随机读和随机写三种方式，这是符合基本事实的。下面对实验中顺序读的异常数据进行分析。

1. 顺序读（read）下块大小（bs）从 4k 升高导致的带宽（BW）断崖下降

当 $\text{bs}=4\text{k}$ 时， $\text{BW}=2095\text{MiB/s}$ ，而当 $\text{bs}=8\text{k}+$ 时，BW 断崖下降至 780MiB/s 左右，这违反了上述数学规律。经过分析，这是因为 $\text{bs}=4\text{k}$ 作为标准 I/O 大小，NVMe SSD 进行了特殊优化。在 NVMe SSD 中，NAND 闪存恰好以 4KB 页为读写单位，恰好与物理页对齐，避免了跨页开销，并通过极高的并行优化调度达到了理论最高带宽。当 $\text{bs}=8\text{k}+$ 时，机侧单次 I/O 的内存拷贝、中断处理以及跨页读取开销占比上升，无法维持 4k 时的超高并发，且单次 I/O 的数据量已经超过单通道的单次读取上限，导致带宽断崖式下降。

2. 顺序读（read）下块大小（bs）在 8k+ 时带宽（BW）的异常稳定

当 bs 从 8k 开始升高时，在 IOPS 基本不变的情况下，BW 却维持平衡而没有按照公式计算结果出现线性增长。经过分析，这是由于 SSD 的 NAND 颗粒已经达到了连续读取的物理极限，继续增大块大小，无法再提升带宽。

(三) 并发 (numjobs) 与队列深度 (iodepth) 对性能的影响

首先需要指出 iodepth 对于 HDD 和 SSD 的影响机制差异：HDD 作为机械硬盘，是顺序访问优先设备，其性能因素在于硬件层面的机械寻道和旋转延迟，此时队列深度通过合并多个随机 I/O 需求，减少寻道次数，提升性能，但是如果队列深度过高，等待延迟会急剧升高并导致性能优化受限；SSD 作为固态硬盘，不依赖于机械部件，队列深度通过打满并行资源，使 SSD 执行多通道 I/O 处理，利用 PCIe 总线带宽，极大优化了随机 I/O 的性能。其次，对于四种读写方式，随着队列深度 (iodepth) 升高，四种读写方式的性能均逐渐升高，但是并发 (numjobs) 对性能的影响不尽相同，对此进行展开分析。

1. 对顺序读和随机写的影响

注意到当 numjobs 由 4 升高至 8 时，顺序读和随机写的性能下降，这是因为顺序读是单流高带宽场景，多线程会导致 I/O 流分裂、PCIe 总线竞争、主机侧调度开销剧增，延迟翻倍；随机写是异步写入 + GC 敏感场景，多线程会导致随机 I/O 请求剧增，触发频繁 GC、磨损均衡，缓存命中率下降，延迟翻倍。所以最优配置是高深度 + 少线程。

2. 对顺序写的影响

注意到当 numjobs 由 4 升高到 8 时，顺序写的 IOPS/BW 小幅提升，但是延迟翻倍。这是因为顺序写是异步写入，多线程能增加缓存写入的请求量，更充分地打满 SLC 缓存带宽，因此 IOPS/BW 小幅提升，但多线程会导致主机侧调度开销增加，延迟翻倍。此时需要权衡延迟和带宽的影响。

3. 对随机读的影响

注意到当 numjobs 由 4 升高至 8 时，随机读的性能大幅提升，这是因为随机读是多流高 IOPS 场景，多线程能提供更多的随机 I/O 请求，让 SSD 的多 Die / 多通道并行资源被完全打满，因此 IOPS/BW 暴涨，延迟因并行处理大幅下降。所以最优配置是高深度 + 多线程。

(四) I/O 引擎对性能的影响

通过折线图分析，我们可以将四种 I/O 引擎对于四种读写方式的性能表现整理为一张表：

表 14: 不同 I/O 引擎在 NVMe SSD 上的性能表现

I/O 引擎	顺序读	顺序写	随机读	随机写
pvsync	次优	最优	最优	最优
libaio	次优	次优	次优	次优
io-uring	最优	最差	次差	次差
mmap	最差	次差	最差	最差

从表中可以看出，pvsync 在 NVMe SSD 中的性能综合表现最好，mmap 性能综合表现最差。

这是因为 pvsync 是伪同步 I/O 引擎，其底层调用异步 I/O 系统 preadv 和 pwritev，但在每个 I/O 操作完成后会主动轮询等待，直到该操作完成才继续下一个，从而模拟出同步 I/O 的顺序执行效果。这样就利用异步 I/O 的底层高效能力，避免传统同步 I/O 在高并发或高 I/O 深度下的性能瓶颈，同时保持测试行为的可预测性。

而 mmap I/O 引擎本质是利用内存映射文件，将文件或设备直接映射到进程的虚拟地址空间，从而允许程序像访问内存一样读写文件。其原理与 NVMe SSD 出现严重冲突，访问映射区域时频繁触发缺页中断，导致延迟开销极大。且 NVMe SSD 擅长高并发、小粒度随机 I/O，而 mmap 的缺页机制本质上是串行化请求，在一个缺页未完成前，进程阻塞在页故障处理中，无法充分利用 SSD 的多队列、多通道并行能力。事实上，mmap 更适用于低速的 HDD 性能优化。

五、 总结

本次实验通过测试不同块大小 (bs)、队列深度 (iodepth) 与并发任务数 (numjobs) 下 NVMe SSD 的顺序读写、随机读写性能，分析了 I/O 参数对存储性能的影响规律，这帮助我从底层学习理解 NVMe SSD 的存储原理和硬件基础。在本次实验中最大的限制因素是如何在 windows 系统中使用 linux 内核完整开展原生 I/O 引擎测试，同时最大程度降低对性能的影响。通过 WSL2 虚拟化技术，我得以在原有硬件的基础上通过 linux 内核开展完整实验。此外，由于计算机组成原理、操作系统等知识的缺失，我也利用课余时间对两者内容进行了实验相关内容学习，这帮助我更好地对实验结果进行底层分析。

下面是实验中借助的 AI 辅助分析截图（共两段对话）：



图 16: AI 辅助分析记录 1



图 17: AI 辅助分析记录 2

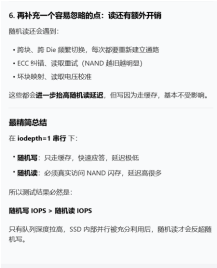


图 18: AI 辅助分析记录 3



图 19: AI 辅助分析记录 4



图 20: AI 辅助分析记录 5

表格		
指标	随块大小增大的变化	底层原因
IOPS	持续下降	IOPS = 每秒完成的 I/O 操作的数量增大，单个数据颗粒减少
BW	持续上升（或趋于饱和）	$BW = IOPS \times bs$ ，块> 增加，抵消了 IOPS 下降硬件瓶颈带宽
latency	持续上升	单次 I/O 需要处理的数据下发到 SSD 完成的时

三、分场景详细解析（计算机系统 + 硬件角度）

1. 顺序读（表 2）

数据规律

• IOPS: 4k(536K) → 8k(99.1K) → 16k(50.2K) → 64k(12.8K), 断崖式下跌

• BW: 4k(2095MB/s) → 8k(774MB/s) → 16k(784MB/s) → 64k(799MB/s), 4k 异常高, 8k + 趋于稳定

• latency: 1.76us → 9.64us → 19.15us → 75.41us, 线性上升

系统 + 硬件解释

• 顺序读的本质: 地址连续, SSD 可以利用预读, 连续顺序行、流水线, 充分发挥 NAND 的连续读取带宽。

• 4k BW 异常高的原因:

1. NVMe 协议 + 主机侧优化: 4k 是操作系统、文件系统的标准块大小, 主机的 IO 调度、DMA、缓存对 4k 操作做了极致优化, 极大提高的并发 (高 iodepth) 打满 PCIe 4.0 x4 的理论带宽 (~20000MB/s 级别) 。

2. SSD 顺序优化: 4k 顺序读是 SSD 的核心测试场景, 固件会开启全量并行 (多 Die、多通道同时工作), 把带宽拉满。

• 8k + BW 骤降并稳定的原因:

1. I/O 调度效率下降: 块大小变大后, 主机侧单次 I/O 的处理开销占比上升, 无法维持 4k 时的超高效开发, 带宽被主机侧瓶颈限制。

2. SSD 并行度饱和: 8k 以上的顺序读, SSD 的 NAND 带宽已经打满, 继续增大块大小, 带宽只会在硬件侧维持小幅波动 (774 → 799MB/s), 无法再提升。

• IOPS 断崖式下跌: 完全符合数学规律, 块大小从 4k → 8k 翻倍, IOPS 从 536K → 99.1K, 下降超过 5 倍, 远大于 2 倍, 说明 4k 时的 IOPS 是主机侧 + 固件优化的“超规格”表现, 8k + 才是 SSD 的真实 IOPS 水平。

2. 顺序写（表 3）

数据规律

• IOPS: 4k(543K) → 8k(363K) → 16k(176K) → 64k(46.7K), 持续下降, 降幅小于顺序读

• BW: 4k(2122MB/s) → 8k(2838MB/s) → 16k(2750MB/s) → 64k(2921MB/s), 持续上升, 最终饱和

• latency: 1.55us → 2.34us → 4.60us → 17.52us, 线性上升

系统 + 硬件解释

• 顺序写的本质: 地址连续, SSD 可以利用 SLC 缓存、DRAM 缓存提前多写入, 先快速填满主机, 后台再写入 TLC/QLC, 同时多通道并行写入。

• BW 持续上升的原因:

1. 带宽效率提升: 块大小越大, 单次写入的数据越多, 缓存的利用率越高, 主机侧的 I/O 开销占比越低, 能更充分地打满 PCIe 带宽。

2. SLC 缓存的顺序写入优化: 顺序写会直接写入 SLC 缓存, 块大小越大, 缓存的写入效率越高, 最终逼近 SLC 缓存的极限带宽 (~30000MB/s 级别) 。

图 21: AI 辅助分析记录 6

2. SSD 顺序优化: 4k 顺序读是 SSD 的核心测试场景, 固件会开启全量并行 (多 Die、多通道同时工作), 把带宽拉满。		
• 8k + BW 骤降并稳定的原因: 1. I/O 调度效率下降: 块大小变大后, 主机侧单次 I/O 的处理开销占比上升, 无法维持 4k 时的超高效开发, 带宽被主机侧瓶颈限制。 2. SSD 并行度饱和: 8k 以上的顺序读, SSD 的 NAND 带宽已经打满, 继续增大块大小, 带宽只会在硬件侧维持小幅波动 (774 → 799MB/s), 无法再提升。		
• IOPS 断崖式下跌: 完全符合数学规律, 块大小从 4k → 8k 翻倍, IOPS 从 536K → 99.1K, 下降超过 5 倍, 远大于 2 倍, 说明 4k 时的 IOPS 是主机侧 + 固件优化的“超规格”表现, 8k + 才是 SSD 的真实 IOPS 水平。		
2. 顺序写（表 3）		
数据规律		
• IOPS: 4k(543K) → 8k(363K) → 16k(176K) → 64k(46.7K), 持续下降, 降幅小于顺序读		
• BW: 4k(2122MB/s) → 8k(2838MB/s) → 16k(2750MB/s) → 64k(2921MB/s), 持续上升, 最终饱和		
• latency: 1.55us → 2.34us → 4.60us → 17.52us, 线性上升		
系统 + 硬件解释		
• 顺序写的本质: 地址连续, SSD 可以利用 SLC 缓存、DRAM 缓存提前多写入, 先快速填满主机, 后台再写入 TLC/QLC, 同时多通道并行写入。		
• BW 持续上升的原因: 1. 带宽效率提升: 块大小越大, 单次写入的数据越多, 缓存的利用率越高, 主机侧的 I/O 开销占比越低, 能更充分地打满 PCIe 带宽。 2. SLC 缓存的顺序写入优化: 顺序写会直接写入 SLC 缓存, 块大小越大, 缓存的写入效率越高, 最终逼近 SLC 缓存的极限带宽 (~30000MB/s 级别) 。		

图 22: AI 辅助分析记录 7

• IOPS 下降幅度小于顺序读: 顺序写的 IOPS 主要受限于缓存写入速度, 而非 NAND 物理速度, 块大小增大时, 缓存的写入效率下降幅度远小于顺序读的 NAND 读取效率下降幅度。		
• 无明显异常: 所有数据完全符合预期, 4k BW 接近 PCIe 带宽, 8k + BW 饱和, IOPS、延迟变化规律正常。		
3. 随机读（表 4）		
数据规律		
• IOPS: 4k(7919) → 8k(7682) → 16k(7464) → 64k(5384), 下降幅度, 64k 才明显下跌		
• BW: 30.9 → 60.0 → 117 → 337MB/s, 线性上升		
• latency: 124.40us → 128.34us → 132.99us → 182.57us, 断崖式上升, 64k 大幅上升		
系统 + 硬件解释		
• 随机读的本质: 地址完全随机, 无法预读, 每次 I/O 都需要真实访问 NAND 颗粒 (读满页 → 读 Die → 读块 → 读页), 延迟由 NAND 的随机读取延迟决定 (~100us 级别) 。		
• IOPS 下降幅度的核心原因: 1. 单址并发性相对受限: 随机读的延迟中, 90% 以上是 NAND 的寻址时间, 数据传输时间占比极低, 块大小从 4k → 16k, 数据量翻 4 倍, 但传输时间仅增加几 us, 总延迟仅从 124us → 133us, 因此 IOPS 几乎不变。 2. 64k IOPS 大幅下跌: 块大小到 64k 时, 数据传输时间占比显著上升, 总延迟从 133us → 182us, IOPS 因此断崖式下降。		
• BW 线性上升: 完全符合数学规律, IOPS 几乎不变, 块大小翻倍, BW 直接翻倍 (30.9 → 60 → 117 → 337, 接近 4k → 8k → 16k → 64k 的倍数关系) 。		
• 无明显异常: 所有数据完全符合 NVMe SSD 随机读的特性, 呈现“标准”的测试结果。		

图 23: AI 辅助分析记录 8

14